

Mixed Precision

Training in 16 bits without losing small updates · video 2:31

This lesson answers one question: how can models train with 16-bit numbers without breaking? 16-bit formats halve memory and speed up math, but **float16** has a small range (tiny gradients underflow to zero) and **bfloat16** has coarse steps (small updates to a weight of 1.0 vanish). **Mixed precision** does the matrix math in 16 bits but keeps float32 master weights: in a tiny GPT it matches float32 (1.645 vs 1.644), while pure bfloat16 weights fall behind (1.677).

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Three formats	▶ 0:08–0:44	2	☐
2. What breaks: rounding and underflow	▶ 0:44–1:21	2	☐
3. Mixed precision in practice	▶ 1:21–2:21	2	☐

Before you start: Deep Dive episodes 18 (backprop) and 19 (AdamW), and Foundations (binary numbers). Code: `code/d22_mixed_precision` (GPT-2 small, about 500 MB, plus three tiny models, about 15 minutes on a CPU).

CONCEPT 1 OF 3

Three formats

▶ Video 0:08-0:44

Most large models are trained with 16-bit numbers instead of 32: half the memory, much faster math on modern GPUs. But 16 bits cannot hold everything. A floating-point number has a sign, an **exponent** (the range) and a **mantissa** (the precision):

FORMAT	BITS (SIGN · EXPONENT · MANTISSA)	LARGEST	STEP AFTER 1.0	SMALLEST NORMAL
float32	1 · 8 · 23	3.4×10^{38}	1.2×10^{-7}	1.2×10^{-38}
float16	1 · 5 · 10	65,504	0.00098	6.1×10^{-5}
bfloat16	1 · 8 · 7	3.4×10^{38}	0.0078 (1/128)	1.2×10^{-38}

32 bits

16 bits

half the memory, much faster math on modern GPUs

but what breaks?

▶ 0:20 32 bits vs 16 bits: half the memory, and much faster math on modern GPUs.

format (bits)

largest value

step after 1.0

smallest normal

float32

float16

bfloat16

3.4×10^{38}

65,504

3.4×10^{38}

1.2×10^{-7}

0.00098

0.0078 (1/128)

1.2×10^{-38}

1.2×10^{-38}

6.1×10^{-5}

▶ 0:42 float32, float16 and bfloat16: largest value, step after 1.0, smallest normal number.

KEY IDEA

float16 trades range for precision; bfloat16 keeps float32's range and gives up precision.

Check 1: Three formats

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 SHORT ANSWER

What do 70,000 become in float16 and in bfloat16?

.....

1.2 CALCULATION

How many GB do the weights of a 7-billion-parameter model take in bfloat16, and in float32?

.....

CONCEPT 2 OF 3

What breaks: rounding and underflow

▶ Video 0:44-1:21

Rounding. Coarse steps swallow small updates. $1.0 + 0.001$ is 1.0010000 in float32, 1.0009766 in float16, and exactly 1.0000000 in bfloat16; $1.0 + 0.0001$ stays 1.0 in both 16-bit formats. A small learning-rate step on a 16-bit weight can simply vanish.

Underflow. A gradient of 10^{-9} becomes exactly 0 in float16. The fix is **loss scaling**: multiply the loss by 1,024 so every gradient is 1,024 times larger, store it, then divide back: 9.9×10^{-10} survives. bfloat16's wide range keeps it (1.0×10^{-9}) without any scaling.

a weight of 1.0 plus a small update

	float32	float16	bfloat16
$1.0 + 0.001$	1.0010000	1.0009766	1.0000000
$1.0 + 0.0001$	1.0001000	1.0000000	1.0000000

red: the update vanished

▶ 0:57 $1.0 + 0.001$: float16 keeps it (1.0009766), bfloat16 rounds it away; $1.0 + 0.0001$: both lose it.

a gradient of 10^{-9}

stored in float16	0.0	
$\times 1024$ in float16, then $\div 1024$	9.9×10^{-10}	loss scaling
stored in bfloat16	1.0×10^{-9}	

▶ 1:20 A gradient of 10^{-9} : 0 in float16; $\times 1024$ then $\div 1024$ rescues it; bfloat16 keeps it.

KEY IDEA

float16 needs loss scaling against underflow; any 16-bit format needs higher-precision weights to keep small updates.

Check 2: What breaks: rounding and underflow

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

In bfloat16, what is $1.0 + 1/256$? And $1.0 + 1/128$?

2.2 TRUE OR FALSE

"Loss scaling is needed with bfloat16 for the same reason as with float16."

True ☐ False ☐

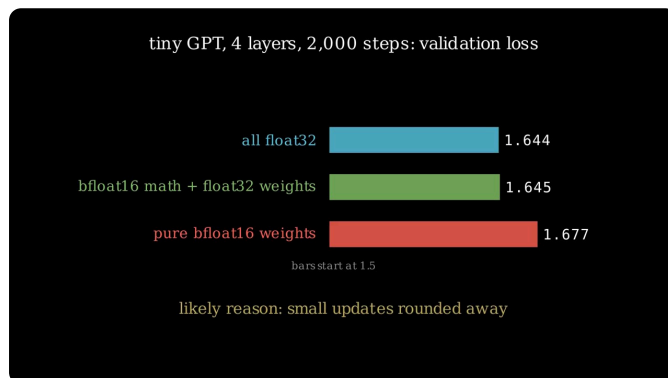
CONCEPT 3 OF 3

Mixed precision in practice

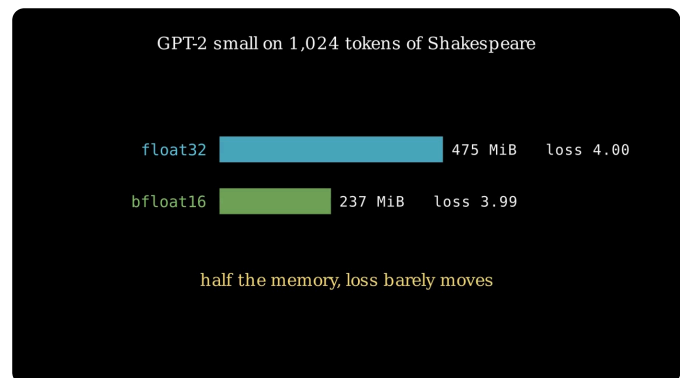
▶ Video 1:21-2:21

Training therefore mixes precisions: the heavy matrix math runs in 16 bits, but a **master copy** of the weights stays in float32, so small updates are not rounded away. The same tiny GPT, 2,000 steps: all float32 **1.644**; bfloat16 math with float32 weights **1.645**, the same; pure bfloat16 weights **1.677**, worse, likely because small updates were rounded away.

For using a model, 16 bits are usually enough: GPT-2 in bfloat16 takes **237 MiB** instead of 475, and its loss on 1,024 tokens of Shakespeare barely moves (3.9897 vs 3.9996). In code, mixed precision is one context manager, `torch.autocast`, around the forward pass; the weights and the optimizer stay in float32.



▶ 1:54 Tiny GPT: float32 1.644; bfloat16 math + float32 weights 1.645; pure bfloat16 weights 1.677.



▶ 2:11 GPT-2 in bfloat16: 237 MiB instead of 475, loss 3.99 instead of 4.00.

KEY IDEA

16-bit math, 32-bit weights: the speed of the first, the accuracy of the second.

Check 3: Mixed precision in practice

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

Which setup matched float32 training in the episode?

- A. Pure bfloat16 weights
- B. bfloat16 math with float32 master weights
- C. float16 weights without loss scaling
- D. None of them

3.2 HANDS-ON CODE

Try it yourself. Check that bfloat16 cannot add 1 to 256.

```
a = torch.tensor(256.0, dtype=torch.bfloat16)
print(a + 1)
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Three formats · rewatch ▶ 0:08-0:44

1.1 float16: infinity (overflow, above 65,504). bfloat16: 70,144 (in range, but rounded).

Checked with torch: float16 has a small range, bfloat16 coarse steps.

1.2 14 GB and 28 GB. 2 and 4 bytes per parameter.

2. What breaks: rounding and underflow · rewatch ▶ 0:44-1:21

2.1 1.0 and 1.0078125. The step after 1.0 is 1/128; half a step rounds back to 1.0.

2.2 False. bfloat16 has float32's exponent range, so tiny gradients do not underflow.

3. Mixed precision in practice · rewatch ▶ 1:21-2:21

3.1 B. 1.645 vs 1.644.

3.2 256. Around 256, bfloat16's step is 2: 257 rounds back to 256.